

# MAIA SPRINT 1

# BUILD SPECIFICATION

v1.1 - Foundation Layer Execution Manual

| Prepared for                           | Date         | Classification | Status                |
|----------------------------------------|--------------|----------------|-----------------------|
| MAIA Business / MAIA Intelligence Core | 18 June 2026 | Confidential   | Ready for Codex Build |

**Purpose**

This document translates the approved Blueprint and Technical Architecture into a concrete Sprint 1 build manual. It tells Codex, Claude Code or a senior developer what to create first, what to avoid and how to validate success. It is the execution bridge between architecture and code.

# Table of Contents

- 01. Executive Summary
- 02. Sprint Objective
- 03. Scope and Non-Goals
- 04. Engineering Principles
- 05. Repository Architecture
- 06. Technology Setup
- 07. Database Foundation
- 08. ERD Final
- 09. Permission Matrix v1
- 10. Prisma Schema Specification
- 11. Multi-Tenant Foundation
- 12. Authentication and Authorization
- 13. Sensitive Access Layer
- 14. Product Context Layer
- 15. Memory Foundation
- 16. Basic Chat Foundation
- 17. API Specification
- 18. Audit and Trace Foundation
- 19. Sprint Task Breakdown
- 20. Acceptance Criteria
- 21. Definition of Done
- 22. Codex Build Instructions
- 23. Sprint 1 Exit Gate

# 01. Executive Summary

Sprint 1 builds the foundation layer of MAIA Intelligence Core. The outcome is not a fully intelligent enterprise operating system yet; it is the secure platform skeleton that allows future intelligence, memory, tools, documents, notifications and product integrations to exist without architectural surgery later. Foundations before chandeliers, because apparently buildings still need floors.

- MAIA Core must be built as an independent central platform, not inside Mikaelson, OFF or Orbit.
- Every request must resolve authentication, tenant, role, permission and product context before accessing data.
- Sensitive access must be protected with a 6-digit code, secure hashing, lockout and temporary sensitive sessions.
- Sprint 1 includes basic chat only; no RAG, embeddings, autonomous tools, voice or fiscal automation.
- The sprint closes only when tenant isolation, permissions, sensitive access, memory persistence and basic chat work end-to-end.

| Artifact                          | Status           | Role in Sprint 1                                  |
|-----------------------------------|------------------|---------------------------------------------------|
| MAIA Intelligence Blueprint v1.1  | Approved         | Product, governance and strategic source of truth |
| MAIA Technical Architecture v1    | Approved         | Technical architecture and engineering rules      |
| Sprint 1 Build Specification v1.1 | Current document | Execution manual for Foundation Layer             |

# 02. Sprint Objective

Deliver a secure, multi-tenant MAIA Core foundation ready for future intelligence layers. Sprint 1 must prove that MAIA can identify who is asking, where they belong, what product context they are using, what they are allowed to see and whether sensitive information is locked or unlocked.

| Objective Area   | Sprint 1 Requirement                                                 | Success Signal                                       |
|------------------|----------------------------------------------------------------------|------------------------------------------------------|
| Identity         | Users authenticate through Clerk and map into internal User records. | Authenticated user can enter the app.                |
| Tenancy          | Tenant and organization context are resolved server-side.            | No data loads without valid tenant scope.            |
| Permissions      | Role and permissions are checked before business logic.              | Unauthorized access returns structured denial.       |
| Sensitive Access | 6-digit code creates temporary sensitive session.                    | Sensitive data remains locked until verification.    |
| Memory           | Memory and MemoryVersion records persist.                            | User can create, update and archive memory.          |
| Chat             | Basic chat validates context and returns a response.                 | Chat works with audit trace and message persistence. |

# 03. Scope and Non-Goals

| Included in Sprint 1                                     | Excluded from Sprint 1                   |
|----------------------------------------------------------|------------------------------------------|
| Repository setup and project structure                   | Voice assistant                          |
| Next.js app shell and dashboard foundation               | WhatsApp, Slack or Gmail integrations    |
| Clerk authentication and organization handling           | RAG, embeddings and vector search        |
| Tenant, organization, user and membership models         | Autonomous agents and Level 5 behavior   |
| Product Context Layer for MAIA, Mikaelson, OFF and Orbit | Fiscal automation and invoice issuing    |
| Sensitive Access Code and Sensitive Session              | Notification Intelligence Engine         |
| Memory and MemoryVersion foundation                      | Cost Engine and advanced usage analytics |
| Basic chat endpoint and UI                               | External tool execution                  |

### Sprint Control Rule

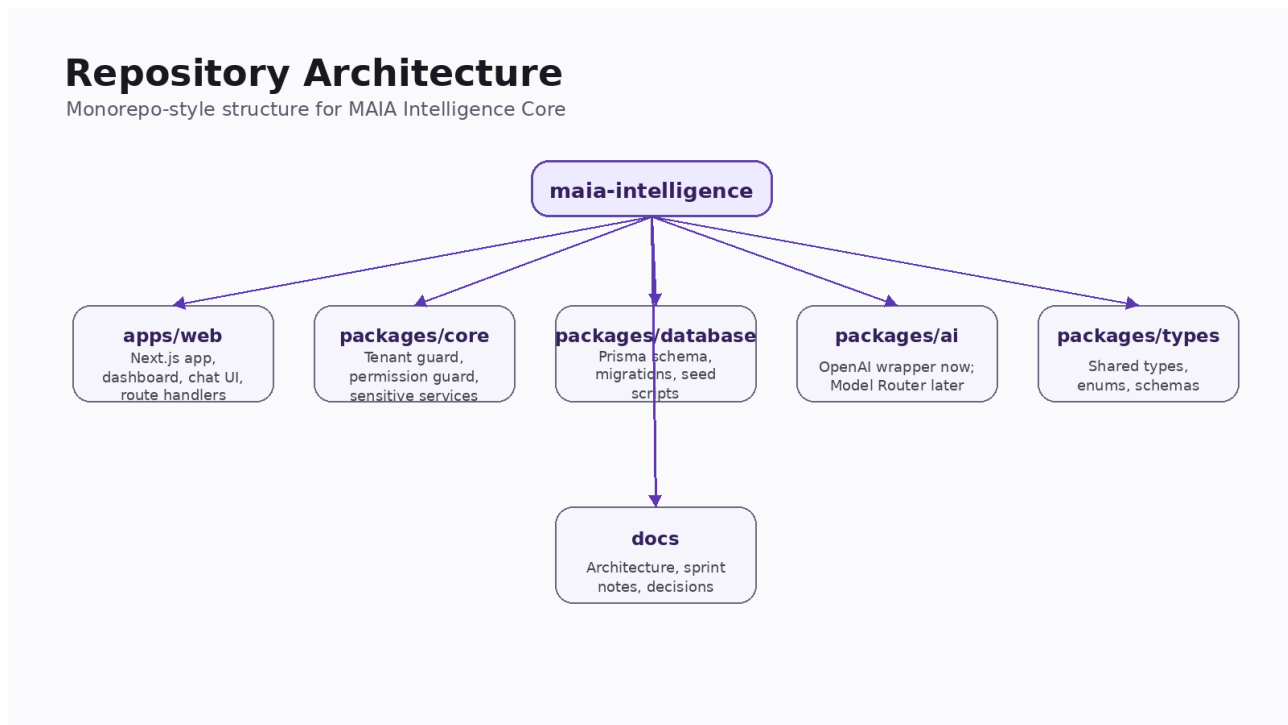
If a feature does not directly support foundation, security, tenancy, sensitive access, memory persistence or basic chat, it does not belong in Sprint 1.

## 04. Engineering Principles

| Principle            | Implementation Meaning                                                                        |
|----------------------|-----------------------------------------------------------------------------------------------|
| Security First       | No AI response or database access bypasses tenant, role, permission or sensitive gates.       |
| Tenant First         | Every scoped query includes tenantId or fails. Super admin paths require audit.               |
| Least Privilege      | Roles receive only the minimum permissions required for their function.                       |
| Auditability         | Every sensitive verification, memory mutation and chat request creates traceable records.     |
| Fail Safe            | When permissions, tenant context or sensitive session state are unclear, deny and log.        |
| No Frontend Trust    | Frontend may hide controls, but backend enforces authorization.                               |
| Future Compatibility | Sprint 1 should not block RAG, tools, notifications or product integrations in later sprints. |

## 05. Repository Architecture

Use a monorepo-style structure that keeps application UI separate from shared core logic, database schema, AI wrappers and shared types. This prevents the usual folder soup where everything goes into /utils and civilization collapses quietly.



| Path          | Purpose                                                                 |
|---------------|-------------------------------------------------------------------------|
| apps/web      | Next.js application, routes, dashboard, chat UI and API route handlers. |
| packages/core | Tenant guard, permission guard, sensitive access services, context      |

|                   |                                                                       |
|-------------------|-----------------------------------------------------------------------|
|                   | resolver and business services.                                       |
| packages/database | Prisma schema, migrations, seed scripts and typed DB client.          |
| packages/ai       | OpenAI wrapper for Sprint 1; later becomes Model Router adapter.      |
| packages/types    | Shared TypeScript types, API contracts, enums and validation schemas. |
| docs              | Architecture docs, sprint notes and engineering decisions.            |

### Repository Naming

Create a private GitHub repository named maia-intelligence. Do not place MAIA Core inside Mikaelson, OFF or Orbit.

## 06. Technology Setup

| Layer          | Selected Technology             | Sprint 1 Use                                                   |
|----------------|---------------------------------|----------------------------------------------------------------|
| Framework      | Next.js App Router + TypeScript | Web app, API routes, server components and streaming-ready UI. |
| Styling        | TailwindCSS                     | Clean admin/product shell styling.                             |
| Authentication | Clerk Organizations             | Users, sessions, organization switching and invitations.       |
| Database       | Neon PostgreSQL                 | Tenant, user, memory, chat and audit records.                  |
| ORM            | Prisma                          | Schema, migrations, typed queries.                             |
| AI             | OpenAI                          | Basic chat response only, routed through packages/ai wrapper.  |
| Storage        | Cloudflare R2                   | Configured but not central until document sprint.              |
| Deployment     | Vercel                          | Preview and production deploys.                                |
| Monitoring     | Sentry + PostHog                | Basic error/product analytics setup if time permits.           |

| Environment Variable              | Required in Sprint 1 | Purpose                                              |
|-----------------------------------|----------------------|------------------------------------------------------|
| DATABASE_URL                      | Yes                  | Prisma database connection.                          |
| DIRECT_URL                        | Recommended          | Direct migration connection for Neon.                |
| CLERK_SECRET_KEY                  | Yes                  | Server-side Clerk authentication.                    |
| NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY | Yes                  | Client-side Clerk configuration.                     |
| OPENAI_API_KEY                    | Yes                  | Basic chat foundation.                               |
| ENCRYPTION_KEY                    | Yes                  | Sensitive payload encryption foundation.             |
| SENSITIVE_PEPPER                  | Yes                  | Extra server-side secret for sensitive code hashing. |
| APP_URL                           | Yes                  | Absolute URL generation and environment consistency. |

## 07. Database Foundation

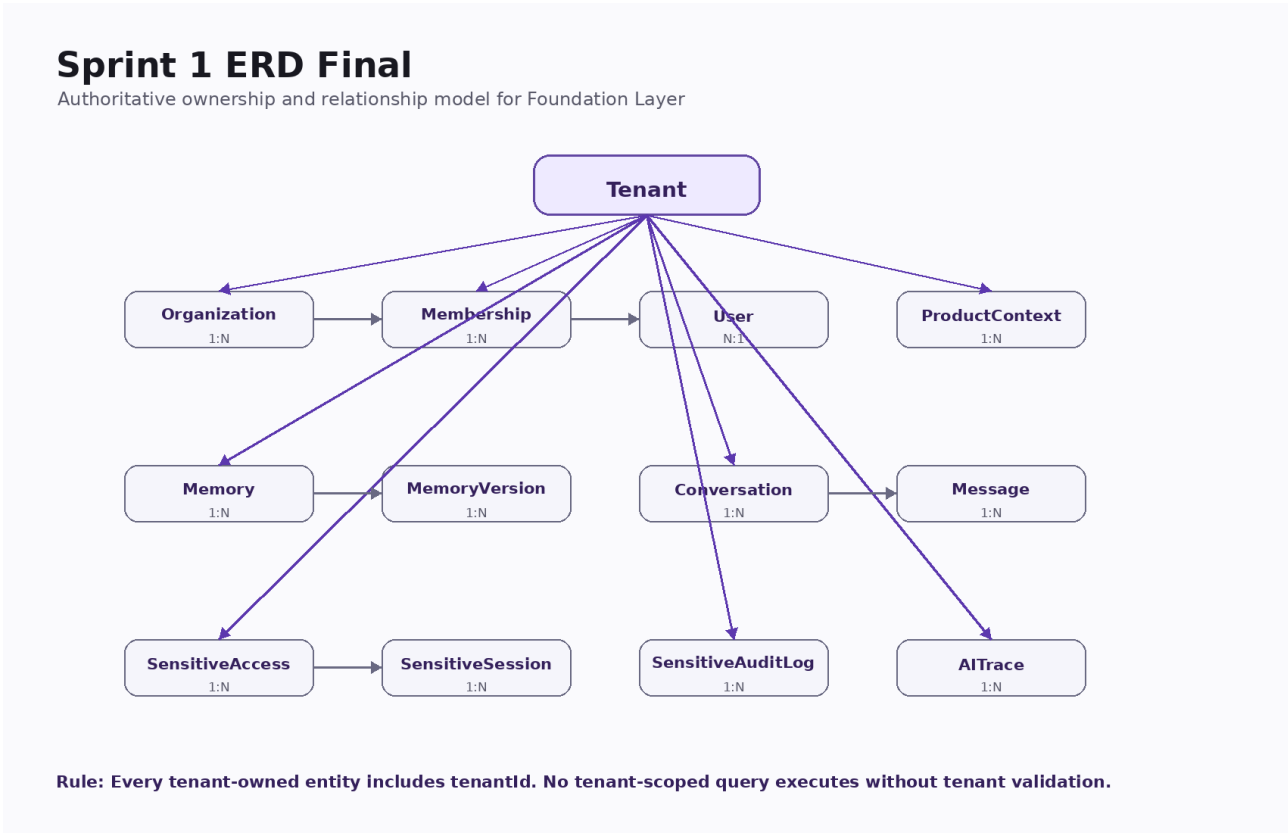
Sprint 1 database design must support tenancy, authentication mapping, product contexts, memory, sensitive sessions, conversations, messages and audit traces. It does not need the full future universe. It needs the foundation clean enough that future tables do not require a bonfire.

| Model        | Purpose                                | Sprint 1 Fields                                    |
|--------------|----------------------------------------|----------------------------------------------------|
| Tenant       | Workspace boundary                     | id, name, slug, status, plan, createdAt, updatedAt |
| Organization | Company/client/community inside tenant | id, tenantId, name, type, status                   |
| User         | Internal user mapped to Clerk          | id, clerkUserId, email, name, status               |
| Membership   | User role inside tenant/org            | id, tenantId, organizationId, userId, role,        |

|                   |                                 |                                                                                    |
|-------------------|---------------------------------|------------------------------------------------------------------------------------|
|                   |                                 | permissions                                                                        |
| ProductContext    | Behavior mode for MAIA products | id, key, name, tone, enabled, limits                                               |
| Memory            | Executive memory record         | id, tenantId, ownerType, ownerId, type, content, securityLevel, confidence, status |
| MemoryVersion     | Memory history                  | id, memoryId, previousContent, newContent, reason, createdBy                       |
| SensitiveAccess   | Sensitive code config           | id, userId, tenantId, codeHash, failedAttempts, lockedUntil                        |
| SensitiveSession  | Temporary unlock                | id, userId, tenantId, mode, expiresAt, revokedAt                                   |
| SensitiveAuditLog | Sensitive access audit          | id, userId, tenantId, action, result, reason, createdAt                            |
| Conversation      | Chat thread                     | id, tenantId, userId, productContext, title, status                                |
| Message           | Chat message                    | id, conversationId, role, content, securityLevel, tokenCount                       |
| AITrace           | AI audit trace                  | id, requestId, tenantId, userId, model, costEstimate, summary                      |

## 08. ERD Final

The following ERD defines the authoritative Sprint 1 data ownership model. It is intentionally limited to Foundation Layer entities. More tables will arrive later; we are not building a database theme park.



| Entity           | Owned By              |
|------------------|-----------------------|
| Tenant           | System                |
| Organization     | Tenant                |
| Membership       | Tenant + Organization |
| User             | Global                |
| ProductContext   | Tenant                |
| Memory           | Tenant                |
| MemoryVersion    | Memory                |
| SensitiveAccess  | User + Tenant         |
| SensitiveSession | SensitiveAccess       |

|                   |              |
|-------------------|--------------|
| SensitiveAuditLog | Tenant       |
| Conversation      | Tenant       |
| Message           | Conversation |
| AITrace           | Tenant       |

| Relationship                        | Cardinality | Rule                                   |
|-------------------------------------|-------------|----------------------------------------|
| Tenant -> Organization              | 1:N         | Tenant owns organizations.             |
| Tenant -> Membership                | 1:N         | Membership determines access.          |
| User -> Membership                  | 1:N         | User may belong to multiple tenants.   |
| Tenant -> Memory                    | 1:N         | Memory is tenant-scoped.               |
| Memory -> MemoryVersion             | 1:N         | Every update creates a version first.  |
| Conversation -> Message             | 1:N         | Messages belong to conversation.       |
| SensitiveAccess -> SensitiveSession | 1:N         | Sessions expire and may be revoked.    |
| Chat Request -> AITrace             | 1:1         | Every response creates trace evidence. |

### Data Isolation Rule

All tenant-owned entities must include tenantId. No query may execute against tenant-scoped data without tenant validation. Cross-tenant joins are prohibited unless explicitly executed through audited administrative flows.

## 09. Permission Matrix v1

Permissions are enforced server-side. Roles define defaults. Permissions define actual capabilities.

### Permission Resolution Flow

Role alone never grants access



| Permission         | Owner | Admin | Manager  | Member   | Client  | Viewer |
|--------------------|-------|-------|----------|----------|---------|--------|
| tenant.read        | Yes   | Yes   | Yes      | Yes      | Limited | Read   |
| tenant.write       | Yes   | Yes   | No       | No       | No      | No     |
| users.read         | Yes   | Yes   | Yes      | No       | No      | No     |
| users.invite       | Yes   | Yes   | Optional | No       | No      | No     |
| memory.read        | Yes   | Yes   | Yes      | Assigned | Own     | Read   |
| memory.write       | Yes   | Yes   | Yes      | Limited  | No      | No     |
| memory.archive     | Yes   | Yes   | Review   | No       | No      | No     |
| conversation.read  | Yes   | Yes   | Yes      | Own      | Own     | Read   |
| conversation.write | Yes   | Yes   | Yes      | Own      | Own     | No     |

|                       |     |             |             |           |            |    |
|-----------------------|-----|-------------|-------------|-----------|------------|----|
| sensitive.read        | Yes | Yes         | Conditional | No        | Own + code | No |
| sensitive.full_access | Yes | Conditional | No          | No        | No         | No |
| audit.read            | Yes | Yes         | Limited     | No        | No         | No |
| cost.read             | Yes | Yes         | Limited     | No        | No         | No |
| actions.execute       | Yes | Yes         | Limited     | Own scope | No         | No |
| admin.system          | Yes | No          | No          | No        | No         | No |

### Authorization Rules

Role alone never grants access. Tenant validation occurs before permission checks. Sensitive permissions require active Sensitive Session. All denials generate audit events. Frontend visibility does not imply authorization.

## 10. Prisma Schema Specification

This section defines the required Sprint 1 database model. Codex must generate Prisma schema based on these specifications.

### Tenant

| Field     | Type     | Constraints                |
|-----------|----------|----------------------------|
| id        | String   | PK                         |
| name      | String   | Required                   |
| slug      | String   | Unique                     |
| status    | String   | Active / Suspended         |
| plan      | String   | Starter / Pro / Enterprise |
| createdAt | DateTime | Default now                |
| updatedAt | DateTime | Auto update                |

### Organization

| Field    | Type   | Constraints                   |
|----------|--------|-------------------------------|
| id       | String | PK                            |
| tenantId | String | FK + indexed                  |
| name     | String | Required                      |
| type     | String | Client / Internal / Community |
| status   | String | Active                        |

### User

| Field       | Type   | Constraints |
|-------------|--------|-------------|
| id          | String | PK          |
| clerkUserId | String | Unique      |
| email       | String | Unique      |
| name        | String | Required    |
| status      | String | Active      |

### Membership

| Field          | Type   | Constraints  |
|----------------|--------|--------------|
| id             | String | PK           |
| tenantId       | String | FK + indexed |
| organizationId | String | FK + indexed |
| userId         | String | FK + indexed |
| role           | Enum   | Required     |
| permissions    | JSON   | Required     |

### ProductContext

| Field | Type | Constraints |
|-------|------|-------------|
|-------|------|-------------|



|         |         |              |
|---------|---------|--------------|
| id      | String  | PK           |
| key     | String  | Unique       |
| name    | String  | Required     |
| tone    | String  | Required     |
| enabled | Boolean | Default true |

## Memory

| Field         | Type   | Constraints                   |
|---------------|--------|-------------------------------|
| id            | String | PK                            |
| tenantId      | String | FK + indexed                  |
| ownerType     | Enum   | User / Org / Product          |
| ownerId       | String | Indexed                       |
| type          | String | Required                      |
| content       | Text   | Required                      |
| confidence    | Int    | 0-100                         |
| securityLevel | Enum   | Public / Internal / Sensitive |
| status        | Enum   | Active / Review / Archived    |

## MemoryVersion

| Field           | Type   | Constraints  |
|-----------------|--------|--------------|
| id              | String | PK           |
| memoryId        | String | FK + indexed |
| previousContent | Text   | Required     |
| newContent      | Text   | Required     |
| reason          | String | Required     |
| createdBy       | String | FK           |

## SensitiveAccess

| Field          | Type     | Constraints  |
|----------------|----------|--------------|
| id             | String   | PK           |
| userId         | String   | FK + indexed |
| tenantId       | String   | FK + indexed |
| codeHash       | String   | Required     |
| failedAttempts | Int      | Default 0    |
| lockedUntil    | DateTime | Nullable     |

## SensitiveSession

| Field     | Type     | Constraints  |
|-----------|----------|--------------|
| id        | String   | PK           |
| userId    | String   | FK + indexed |
| tenantId  | String   | FK + indexed |
| mode      | Enum     | Read / Full  |
| expiresAt | DateTime | Required     |
| revokedAt | DateTime | Nullable     |

## Conversation

| Field          | Type   | Constraints  |
|----------------|--------|--------------|
| id             | String | PK           |
| tenantId       | String | FK + indexed |
| userId         | String | FK + indexed |
| productContext | String | Indexed      |
| title          | String | Nullable     |
| status         | String | Active       |

## Message

| Field          | Type   | Constraints                   |
|----------------|--------|-------------------------------|
| id             | String | PK                            |
| conversationId | String | FK + indexed                  |
| role           | Enum   | User / Assistant              |
| content        | Text   | Required                      |
| tokenCount     | Int    | Nullable                      |
| securityLevel  | Enum   | Public / Internal / Sensitive |

## AITrace

| Field         | Type    | Constraints  |
|---------------|---------|--------------|
| id            | String  | PK           |
| requestId     | String  | Indexed      |
| tenantId      | String  | FK + indexed |
| userId        | String  | FK + indexed |
| modelProvider | String  | Required     |
| modelName     | String  | Required     |
| costEstimate  | Decimal | Nullable     |
| summary       | Text    | Required     |

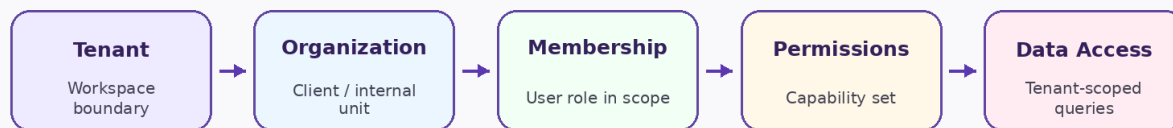
### Database Standards

Every tenant-owned entity includes tenantId. Every foreign key must be indexed. Soft-delete is preferred over hard-delete. Sensitive data must never be stored in plaintext. Audit records are immutable. Schema changes require migration review before production deployment.

## 11. Multi-Tenant Foundation

### Multi-Tenant Access Model

Every query must resolve tenant and membership before data access



| Tenant Rule         | Implementation Requirement                                                |
|---------------------|---------------------------------------------------------------------------|
| Tenant required     | All tenant-scoped operations must resolve tenantId before business logic. |
| Membership required | User must have active Membership for the tenant.                          |

|                        |                                                                               |
|------------------------|-------------------------------------------------------------------------------|
| Role per tenant        | Same user may have different roles in different tenants.                      |
| No cross-tenant memory | Memory retrieval filters tenantId, ownerId, securityLevel and productContext. |
| Super admin exception  | Allowed only through explicit audited admin flow.                             |
| Backend enforcement    | Frontend visibility is UX only; backend owns access decisions.                |

| Function            | Responsibility                                                 | Fail Behavior                          |
|---------------------|----------------------------------------------------------------|----------------------------------------|
| resolveTenant()     | Determine tenant from session, route or selected organization. | Return 400/403 and log if missing.     |
| requireMembership() | Verify active user membership.                                 | Return 403 if absent/inactive.         |
| requirePermission() | Verify permission before action.                               | Return 403 with structured error.      |
| tenantWhere()       | Inject tenantId into Prisma queries.                           | Do not execute query without tenantId. |

## 12. Authentication and Authorization

| Role    | Typical User       | Sprint 1 Access                                       |
|---------|--------------------|-------------------------------------------------------|
| OWNER   | MAIA owner/founder | Full tenant administration and setup.                 |
| ADMIN   | Tenant admin       | Manage users, context, memory and sensitive setup.    |
| MANAGER | Team lead          | Operational read/write inside permitted scope.        |
| MEMBER  | Internal user      | Chat, assigned memory and limited workspace.          |
| CLIENT  | External client    | Limited client portal data and own sensitive profile. |
| VIEWER  | Stakeholder        | Read-only access.                                     |

### Authorization Rule

Permissions are enforced server-side. The frontend may hide controls, but the backend decides. Trusting JavaScript as security is not engineering; it is theater with syntax highlighting.

# 13. Sensitive Access Layer

## Sensitive Access Flow

Sensitive information is never unlocked by normal authentication alone



| Control            | Sprint 1 Decision                                                             |
|--------------------|-------------------------------------------------------------------------------|
| Code format        | 6-digit numeric code.                                                         |
| Storage            | Never store plain code; hash with Argon2id or bcrypt plus SENSITIVE_PEPPER.   |
| Failed attempts    | 5 attempts before lockout.                                                    |
| Lockout            | 15 minutes.                                                                   |
| Session mode       | read or full.                                                                 |
| Session expiration | 15 minutes default; configurable later.                                       |
| Audit              | Every setup, verification, failure and revocation is logged.                  |
| Display            | Sensitive values should be masked unless full display is explicitly required. |

| Endpoint                           | Purpose                                                 | Validation                           |
|------------------------------------|---------------------------------------------------------|--------------------------------------|
| POST /api/v1/sensitive/setup-code  | Create or update sensitive code.                        | Auth + tenant + secure confirmation. |
| POST /api/v1/sensitive/verify-code | Create sensitive session after successful verification. | Auth + tenant + lockout policy.      |
| POST /api/v1/sensitive/revoke      | Revoke active sensitive session.                        | Auth + tenant + session ownership.   |

# 14. Product Context Layer

| Context   | Sprint 1 Behavior                       | Future Behavior                |
|-----------|-----------------------------------------|--------------------------------|
| maia      | General MAIA Core mode.                 | Admin and strategy operations. |
| mikaelson | Executive/operational tone placeholder. | Projects, KPIs, tasks, risks.  |
| off       | Editorial/community tone placeholder.   | Chapters, members, content.    |
| orbit     | Fiscal/security tone placeholder.       | RFC, CFDI, invoices, tickets.  |

| Context Package Field | Required        | Description                          |
|-----------------------|-----------------|--------------------------------------|
| tenantId              | Yes             | Tenant boundary for all operations.  |
| organizationId        | When applicable | Organization/client/community scope. |

|                      |          |                                 |
|----------------------|----------|---------------------------------|
| userId               | Yes      | Authenticated internal user ID. |
| productContext       | Yes      | maia, mikaelson, off or orbit.  |
| permissions          | Yes      | Server-resolved permissions.    |
| sensitiveSessionMode | Yes      | none, read or full.             |
| conversationId       | Optional | Existing chat thread.           |

## 15. Memory Foundation

Sprint 1 builds memory persistence and governance basics. It does not build embeddings or RAG yet. MAIA must store strategic memory, not every random chat fragment humanity produces while avoiding decisions.

| Memory Operation | Sprint 1 Behavior                                                      | Audit Required |
|------------------|------------------------------------------------------------------------|----------------|
| Create           | Create memory with owner, type, content, confidence and securityLevel. | Yes            |
| Update           | Create MemoryVersion before changing active content.                   | Yes            |
| Archive          | Remove from active retrieval but preserve record.                      | Yes            |
| Delete           | Not default; restricted to admin/user request/compliance.              | Yes            |
| Review           | Mark low-confidence or sensitive memories for review.                  | Yes            |

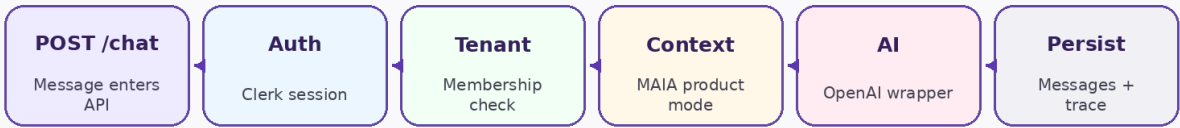
| Factor             | Initial Weight | Meaning                                                       |
|--------------------|----------------|---------------------------------------------------------------|
| Importance         | 35%            | Business relevance, future usefulness and operational impact. |
| Confidence         | 25%            | Source reliability, explicitness and corroboration.           |
| Sensitivity        | 20%            | Fiscal, legal, personal, contract or financial content.       |
| Recency            | 10%            | Newer facts weigh more unless older fact remains active.      |
| Conflict Detection | 10%            | Contradiction with active memory triggers version/review.     |

| Final Score | Action                           |
|-------------|----------------------------------|
| 80-100      | Store automatically.             |
| 60-79       | Store and mark for review.       |
| Below 60    | Discard as conversational noise. |

# 16. Basic Chat Foundation

## Basic Chat Request Sequence

Sprint 1 chat has no RAG, no tools and no autonomous execution



Sprint 1 chat is intentionally simple. It validates the request, assembles a minimal ContextPackage, calls OpenAI through the packages/ai wrapper, persists messages and writes AITrace. No RAG. No tools. No automation. The chatbot is not allowed to cosplay as a COO yet.

| Step                     | Behavior                                                                         |
|--------------------------|----------------------------------------------------------------------------------|
| 1. Receive request       | POST /api/v1/chat receives tenantId, productContext, conversationId and message. |
| 2. Validate auth         | Clerk session must map to internal User.                                         |
| 3. Validate tenant       | User must have active Membership.                                                |
| 4. Resolve context       | Build ContextPackage from user, tenant, permissions and productContext.          |
| 5. Check sensitive state | Block sensitive content if no valid sensitive session.                           |
| 6. Call AI               | Use packages/ai wrapper, not direct UI model call.                               |
| 7. Persist               | Save user and assistant messages.                                                |
| 8. Trace                 | Create AITrace with model, requestId, tenantId and summary.                      |

# 17. API Specification

| Endpoint                           | Purpose                       | Sprint 1 Security                     |
|------------------------------------|-------------------------------|---------------------------------------|
| POST /api/v1/chat                  | Basic contextual chat.        | Auth + Tenant Guard + Product Context |
| GET/POST /api/v1/conversations     | List/create conversations.    | Auth + tenant scoped                  |
| GET/POST /api/v1/memory            | Read/create memory.           | memory.read/write                     |
| PATCH /api/v1/memory/:id           | Update/archive memory.        | memory.write + audit                  |
| POST /api/v1/sensitive/setup-code  | Create/update sensitive code. | Auth + tenant + confirmation          |
| POST /api/v1/sensitive/verify-code | Create sensitive session.     | Lockout + audit                       |
| GET /api/v1/audit                  | Admin trace review.           | admin.audit                           |

| API Contract      | Request                                                | Response                              |
|-------------------|--------------------------------------------------------|---------------------------------------|
| POST /api/v1/chat | { tenantId, productContext, conversationId?, message } | { response, conversationId, traceId } |

|                                    |                                                                |                                |
|------------------------------------|----------------------------------------------------------------|--------------------------------|
| POST /api/v1/sensitive/verify-code | { tenantId, code, mode }                                       | { sessionId, mode, expiresAt } |
| POST /api/v1/memory                | { tenantId, ownerType, ownerId, type, content, securityLevel } | { memoryId, status }           |

### API Contract Principles

All product-facing APIs use /api/v1/\*. Every endpoint validates auth, tenant scope and permissions before business logic. Structured errors must include code, message and requestId where applicable.

## 18. Audit and Trace Foundation

| Event                    | When Logged              | Minimum Fields                              |
|--------------------------|--------------------------|---------------------------------------------|
| auth.login               | User signs in.           | userId, tenantId?, timestamp                |
| tenant.access_denied     | Tenant validation fails. | userId, attemptedTenantId, reason           |
| memory.created           | Memory is created.       | memoryId, userId, tenantId, securityLevel   |
| memory.updated           | Memory content changes.  | memoryId, versionId, reason                 |
| sensitive.verify_success | Sensitive code succeeds. | userId, tenantId, sessionId, mode           |
| sensitive.verify_failed  | Sensitive code fails.    | userId, tenantId, failedAttempts            |
| chat.request             | Chat request starts.     | requestId, userId, tenantId, productContext |
| chat.response            | Chat response completes. | requestId, model, tokenEstimate, traceId    |

| AITrace Field           | Sprint 1 Purpose                                 |
|-------------------------|--------------------------------------------------|
| requestId               | Connect chat request, messages and audit record. |
| tenantId/userId         | Scope trace to tenant and user.                  |
| productContext          | Show which product mode was used.                |
| modelProvider/modelName | Identify model route.                            |
| permissionsEvaluated    | Record access decisions.                         |
| sensitiveSessionUsed    | Show whether sensitive access affected response. |
| confidenceSummary       | Safe summary, not hidden chain-of-thought.       |
| costEstimate            | Basic usage tracking placeholder.                |

## 19. Sprint Task Breakdown

| Epic | Build Area        | Key Tasks                                                         | Exit Criteria                         |
|------|-------------------|-------------------------------------------------------------------|---------------------------------------|
| 01   | Repository Setup  | Create repo, install Next.js, TypeScript, Tailwind, base folders. | App runs locally and deploys preview. |
| 02   | Database Setup    | Configure Prisma, Neon, schema models and migrations.             | Migration succeeds and seed works.    |
| 03   | Auth Layer        | Install Clerk, protect routes, sync user to DB.                   | Authenticated user reaches dashboard. |
| 04   | Tenant Layer      | Create tenant/org/membership resolver and guards.                 | Cross-tenant tests deny access.       |
| 05   | Product Context   | Create context enum, resolver and request validation.             | Chat accepts productContext safely.   |
| 06   | Sensitive Access  | Create code setup, verify, session, logout and audit.             | Code flow works with expiry.          |
| 07   | Memory Foundation | CRUD memory, versioning, archive and review status.               | Memory persists and versions update.  |
| 08   | Chat Foundation   | Create chat UI, endpoint, messages and AITrace.                   | User can chat with trace.             |
| 09   | QA + Hardening    | Security tests, tenant tests, sensitive tests, deployment.        | Sprint accepted.                      |

## 20. Acceptance Criteria

| Area             | Acceptance Criteria                                                         |
|------------------|-----------------------------------------------------------------------------|
| Authentication   | Users can sign in/out. Protected routes cannot be accessed unauthenticated. |
| Tenant Isolation | User cannot read data from tenant where they lack membership.               |
| Authorization    | Permission checks happen server-side before business logic.                 |
| Sensitive Access | Code setup, failed attempts, lockout, verify and session expiry work.       |
| Memory           | Memory create, update, version and archive work inside tenant scope.        |
| Chat             | Chat endpoint validates tenant/context and persists conversation/messages.  |
| Audit            | Sensitive and chat operations produce traceable audit records.              |
| Deployment       | Preview deployment works and production deployment can be promoted.         |

## 21. Definition of Done

| Gate               | Required Result                                                |
|--------------------|----------------------------------------------------------------|
| Security Gate      | No critical tenant, permission or sensitive access failure.    |
| Code Quality Gate  | TypeScript passes; lint passes; no direct model calls from UI. |
| Database Gate      | Prisma migration reviewed and applied successfully.            |
| Tenant Gate        | All tenant-scoped queries include tenant validation.           |
| Sensitive Gate     | Sensitive sessions expire and failed attempts lock correctly.  |
| Audit Gate         | Critical operations generate trace records.                    |
| Deployment Gate    | Vercel preview works and environment variables are documented. |
| Documentation Gate | README and Sprint 1 notes updated.                             |

## 22. Codex Build Instructions

### Primary Instruction

Codex must implement Sprint 1 only. It must not add RAG, embeddings, voice, WhatsApp, autonomous agents, fiscal automation, external tool execution or advanced notifications.

| Instruction Area | Codex Requirement                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------|
| Scope Discipline | Build only the Foundation Layer defined in this specification.                                    |
| Security         | Implement auth, tenant guard, permission guard and sensitive gate before chat behavior.           |
| Database         | Create Prisma schema and migrations for Sprint 1 models only.                                     |
| Architecture     | Follow repository structure exactly. Do not collapse everything into one utils folder.            |
| API              | Create /api/v1 endpoints with structured validation and errors.                                   |
| Testing          | Include basic tests or test notes for tenant, sensitive and memory flows.                         |
| No Assumptions   | If a requirement is ambiguous, leave a TODO with a clear question rather than inventing behavior. |

### Codex Master Prompt

You are a senior full-stack software engineer building MAIA Intelligence Core Sprint 1. Implement only the Foundation Layer described in this specification.



Repository: maia-intelligence

Stack: Next.js App Router, TypeScript, TailwindCSS, Prisma, Neon PostgreSQL, Clerk Organizations, OpenAI wrapper, Vercel.

Build order:

1. Repository and app shell.
2. Prisma schema and migrations for Sprint 1 models.
3. Clerk authentication and internal User sync.
4. Tenant, organization and membership guards.
5. Product Context resolver.
6. Sensitive Access Code, lockout, Sensitive Session and SensitiveAuditLog.
7. Memory CRUD with MemoryVersion before updates.
8. Basic chat endpoint and UI with AITrace.

Strict exclusions: no RAG, embeddings, vector search, voice, WhatsApp, Gmail, autonomous agents, fiscal automation, notifications, cost engine or external tool execution.

Security requirements: every endpoint validates authentication, tenant scope and permissions before business logic. Sensitive data requires an active Sensitive Session. No direct model calls from UI. No tenant-scoped query without tenantId.

If a requirement is ambiguous, create a TODO and ask; do not invent architecture.

## 23. Sprint 1 Exit Gate

| Gate                 | Status Required                                   |
|----------------------|---------------------------------------------------|
| MAIA Core Repository | Created and running locally.                      |
| Foundation Schema    | Implemented and migrated.                         |
| Authentication       | Working through Clerk.                            |
| Tenant Guard         | Enforced in server logic.                         |
| Sensitive Access     | Code, session, lockout and audit working.         |
| Memory Foundation    | Create/update/archive/version working.            |
| Basic Chat           | Working with safe context and trace.              |
| Sprint 2 Readiness   | Document and RAG work can begin without redesign. |

### Final CTO Directive

Sprint 1 succeeds when MAIA can answer inside the correct tenant, with correct permissions, locked sensitive data, persistent memory and audit visibility. Anything beyond that belongs to later sprints. Architecture is done. Now make the thing exist.